

On-Device Classification of Lighting Conditions Using an ESP32

Pina Cammisano

October 2026

1 Proposed Pipeline

For this project, I built a system that can identify four lighting conditions: LED, fluorescent, shade, and sunlight. The system uses an ESP32 connected to a GUVVA-S12SD UV sensor and a TSL2591 light sensor.

The main goal was to train a classification model on a computer and then run it directly on the ESP32. This way, the ESP32 can make predictions without needing a computer or an internet connection.

The model was trained using DBSCAN and Kernel Density Estimation (KDE). DBSCAN was used to find groups in the sensor data without looking at the class labels. KDE was then used to estimate how the readings were spread out inside each group.

The KDE results were stored in four small tables. Each table contained a 16×16 grid of unsigned 8-bit values. This reduced the amount of memory needed for the model.

During live testing, the ESP32 used an Exponential Moving Average (EMA) filter to smooth the sensor readings. The filtered values were then used for classification.

A stored table value was converted back into a number between 0 and 1 using

$$\tilde{g}_c(\mathbf{x}_{ij}) = \frac{q_{ij}}{255}. \quad (1)$$

I used nearest-cell lookup because it is simple and only requires one table value from each class.

Algorithm 1 shows how the ESP32 classifies a new sensor reading.

2 Application

The purpose of this system is to identify different types of lighting using two sensor readings.

Algorithm 1 Lighting Classification

Require: UV reading x and IR reading y

Require: Stored class tables Q_c

Require: Training scaling bounds

Require: Grid size G and threshold τ

Ensure: Predicted class or UNKNOWN

```
1: Apply EMA filtering to the sensor readings
2:  $y \leftarrow \log(1 + y)$ 
3:  $\tilde{x} \leftarrow \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ 
4:  $\tilde{y} \leftarrow \frac{y - y_{\min}}{y_{\max} - y_{\min}}$ 
5: Clip both values to  $[0, 1]$ 
6:  $\text{col} \leftarrow \lfloor \tilde{x}G \rfloor$ 
7:  $\text{row} \leftarrow \lfloor \tilde{y}G \rfloor$ 
8: Clip row and column to 0 through  $G - 1$ 
9: for each class  $c$  do
10:    $v_c \leftarrow Q_c[\text{row}, \text{col}] / 255$ 
11: end for
12:  $\hat{c} \leftarrow \arg \max_c v_c$ 
13: if  $v_{\hat{c}} < \tau$  then
14:   return UNKNOWN
15: end if
16: return  $\hat{c}$ 
```

The UV sensor provides an analog measurement, while the TSL2591 provides the IR reading. The ESP32 uses both values to decide which lighting condition best matches the measurement.

I wanted the classification to happen directly on the ESP32 instead of sending every reading to my computer.

To do this, I trained the model using Python and converted the results into small lookup tables.

When a new measurement is received, the ESP32 scales the values, finds the matching grid cell, and compares the stored values for all four classes.

This method is useful for embedded systems because it requires very little memory and processing

time.

The model also uses a reject threshold. If the highest stored value is below this threshold, the system returns UNKNOWN rather than selecting one of the four classes.

3 Method and Setup

3.1 Hardware Setup

For the hardware, I used an ESP32 development board, a GUVVA-S12SD UV sensor, and a TSL2591 light sensor.

The UV sensor was connected to GPIO34 on the ESP32.

The TSL2591 used I2C communication. Its SDA pin was connected to GPIO21, and its SCL pin was connected to GPIO22.

Both sensors were powered using 3.3 V.

The TSL2591 was configured with medium gain and a 100 ms integration time.

The program used a target sampling interval of 100 ms.

Figure 1 shows the circuit used during testing.

3.2 Data Collection

I collected sensor readings under four lighting conditions: LED, fluorescent, shade, and sunlight.

I collected 600 labeled measurements for each lighting condition.

In total, the dataset contained

$$4 \times 600 = 2400$$

samples.

Each sample contained a UV reading, an IR reading, and the lighting label.

I divided the dataset into training, validation, and testing sets using a fixed random seed of 20250903.

The split was

60% training, 20% validation, 20% testing.

This gave me 1440 training samples, 480 validation samples, and 480 test samples.

Each lighting class had 360 training samples, 120 validation samples, and 120 test samples.

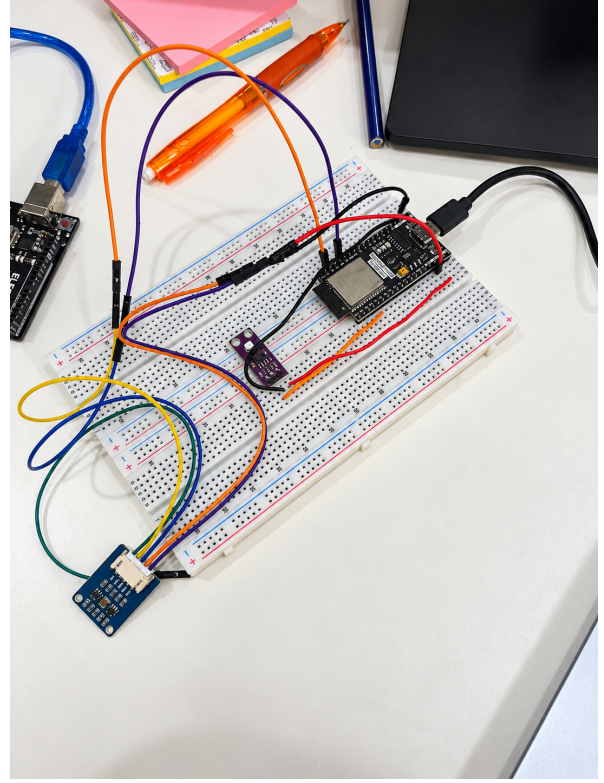


Figure 1: ESP32 connected to the UV and TSL2591 sensors.

The training set was used to build the model. The validation set was used to choose the KDE bandwidth. The test set was kept separate for the final evaluation.

3.3 EMA Filtering

During live testing, I used an Exponential Moving Average (EMA) filter to smooth the sensor readings.

Sensor values can change slightly even when the lighting stays the same. These small changes can introduce noise into the measurements.

EMA helps reduce this noise by combining the current reading with the previous filtered value.

The equation is

$$y_n = \alpha x_n + (1 - \alpha)y_{n-1}. \quad (2)$$

Here, x_n is the current raw reading, y_n is the new filtered reading, and y_{n-1} is the previous filtered

reading.
I used

$$\alpha = 0.2.$$

This means that the new filtered value uses 20% of the current reading and 80% of the previous filtered value.

I chose EMA because it is simple to implement and only needs the previous filtered value. It does not require storing a large number of earlier samples.

This is useful on the ESP32 because it requires very little memory.

One disadvantage is that EMA reacts more slowly when the lighting changes suddenly. This can be seen in the filtering results.

3.4 Feature Scaling

Before applying DBSCAN, I had to prepare the two sensor features.

The UV and IR readings had different numerical ranges.

If I used the original values directly, the feature with the larger range could have too much influence on the distance calculations.

I first applied a logarithmic transformation to the IR readings:

$$IR_{\log} = \log(1 + IR). \quad (3)$$

This reduced the difference between small and large IR values.

I then used min-max scaling to place both features between 0 and 1.

The minimum and maximum values were calculated using the training data.

The same scaling values were included in the ESP32 model so that new measurements would be processed the same way.

3.5 DBSCAN Clustering

I used DBSCAN to find groups in the training data without using the labels.

The final parameters were

$$\epsilon = 0.03, \quad \text{min_samples} = 8.$$

After applying the logarithmic IR transformation and min-max scaling, DBSCAN found four clusters.

Table 1: Final DBSCAN results.

Measurement	Result
Training samples	1440
Epsilon	0.03
Minimum samples	8
Clusters	4
Samples per cluster	360
Noise points	0

Table 2: DBSCAN clusters compared with actual lighting labels.

Actual class	C0	C1	C2	C3
LED	360	0	0	0
Fluorescent	0	360	0	0
Shade	0	0	360	0
Sun	0	0	0	360

Each cluster contained 360 samples, and there were no noise points.

Table 1 shows the results.

The clusters also matched the four known lighting labels.

Table 2 compares the DBSCAN clusters with the original labels.

This was an improvement over my earlier tests, where some lighting conditions were grouped together.

The biggest difference came from changing how I prepared the sensor readings.

Once I used the logarithmic transformation and scaling, DBSCAN was able to separate all four groups.

DBSCAN did not use the labels when finding these clusters. The labels were only used afterward to check the results.

3.6 KDE and Model Generation

After finding the clusters, I used Gaussian Kernel Density Estimation (KDE) to model each group.

KDE estimates where the measurements are most concentrated.

I tested different bandwidth values using the validation set.

Among the values I tested, the best bandwidth was

Table 3: Final classification test results.

Class	Tested	Correct
LED	120	120
Fluorescent	120	120
Shade	120	120
Sun	120	120
Total	480	480

$$h = 0.03.$$

It gave a mean validation log-likelihood of approximately 5.122.

The final model used

$$G = 16, \quad h = 0.03, \quad \tau = 0.12.$$

The four KDE surfaces were stored in 16×16 grids.

Each value was converted into an unsigned 8-bit number between 0 and 255.

I used a Python script to generate the `model.h` file.

This file contained the four class tables, scaling bounds, class names, grid size, and reject threshold.

The ESP32 could then use this model without running DBSCAN or KDE calculations itself.

Figure 2 compares the continuous KDE surfaces with the stored 8-bit versions.

The stored surfaces have less detail because they only contain 16×16 values.

However, the main high-density regions remain visible in the stored tables.

4 Results and Evidence

4.1 Classification Results

I tested the final model using 480 samples that were kept separate from training.

All 480 samples were classified correctly according to their recorded labels.

Table 3 shows the test results.

The test accuracy was

$$\frac{480}{480} \times 100 = 100\%.$$

Table 4: Storage for four 16×16 tables.

Format	Bytes/cell	Total
uint8	1	1024 B
uint16	2	2048 B
float32	4	4096 B
float64	8	8192 B

I also compared the stored 8-bit model with the continuous KDE model.

Both models made the same predictions for all 480 test samples.

The agreement was therefore also 100%.

This showed that converting the KDE surfaces into small lookup tables did not change the predictions on this test set.

However, these samples came from the same data collection sessions.

I would need to test the model under different lighting conditions and on different days to know how well it works in new environments.

4.2 Model Storage

The model had four classes. Each class used a 16×16 lookup table.

Each stored value required one byte.

The table storage was

$$\begin{aligned} \text{Storage} &= K \cdot G^2 \cdot b \\ &= 4 \cdot 16^2 \cdot 1 \\ &= 1024 \text{ bytes.} \end{aligned} \tag{4}$$

The four scaling bounds required another 16 bytes.

This gave

$$1024 + 16 = 1040 \text{ bytes.}$$

This calculation includes the four class tables and scaling bounds, not the complete program.

Table 4 compares different storage formats.

I selected uint8 because it required less memory than the other formats.

The largest measured quantization error was approximately 0.189% of the peak value.

The stored model still made the same predictions as the continuous KDE model for all 480 held-out test samples.

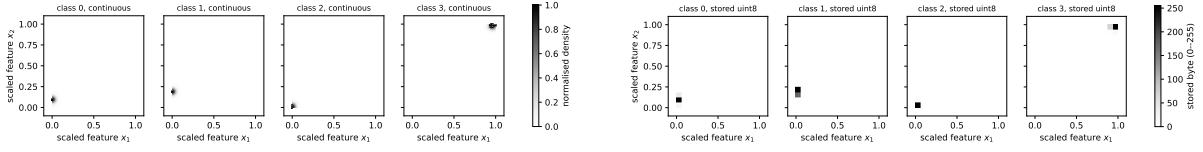


Figure 2: Continuous KDE surfaces and their stored 8-bit versions for the four lighting classes. Classes 0, 1, 2, and 3 represent LED, fluorescent, shade, and sun.

Table 5: Measured ESP32 memory usage.

Memory	Used	Percent
Flash	308800 B	23%
SRAM	23740 B	7%

This showed that the memory saving did not affect the predictions on the test data.

4.3 ESP32 Memory Usage

I compiled the final Arduino program to measure how much memory it required.

The compiler reported 308800 bytes of program storage, which was 23% of the available flash memory.

The global variables used 23740 bytes of SRAM, which was 7% of the available dynamic memory.

Table 5 shows the memory results.

These measurements include the complete program and its libraries, not just the lookup tables.

The results show that the program fits within the ESP32’s memory limits, with space remaining for other operations.

4.4 Inference Time

I measured how long the ESP32 took to classify a sensor reading.

I used the Arduino `micros()` function to measure the time needed for classification.

During testing, the Serial Monitor showed inference times of approximately 3–6 microseconds.

This measurement included the classification operation.

It did not include the time required to collect the sensor readings or print the results.

The short inference time shows why using a lookup table is useful.

Instead of performing KDE calculations for each reading, the ESP32 only needs to scale the values and compare four stored table values.

4.5 ESP32 and Python Comparison

To check the ESP32 implementation, I compared ten readings from the device with the Python classifier.

I used the same UV and IR measurements in both versions and compared their predicted classes.

All ten predictions matched.

The agreement was

$$\frac{10}{10} \times 100 = 100\%.$$

This showed that the ESP32 and Python implementations produced the same results for these selected measurements.

However, ten readings are not enough to test every possible lighting condition or sensor value.

5 EMA Filtering Results

I recorded raw and EMA-filtered IR readings under the four lighting conditions.

I used the values visible in the Serial Monitor screenshots to create CSV files.

I then used Python to plot the raw and filtered readings.

The horizontal axis shows the sample number, and the vertical axis shows the IR sensor reading.

These graphs contain the readings visible in the screenshots rather than complete continuous recordings.

5.1 Shade

Figure 3 compares the raw and filtered readings under shade.

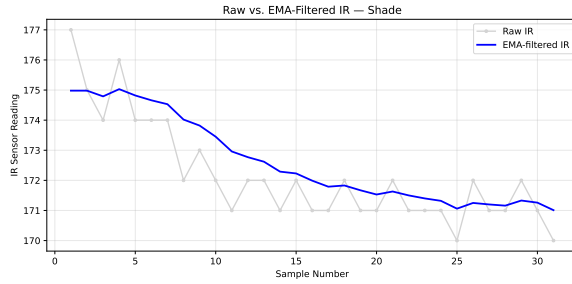


Figure 3: Raw and EMA-filtered IR readings under shade.

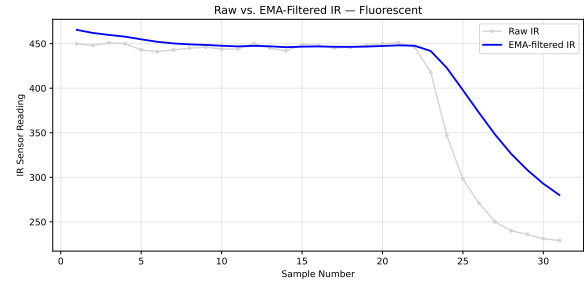


Figure 5: Raw and EMA-filtered IR readings under fluorescent lighting.

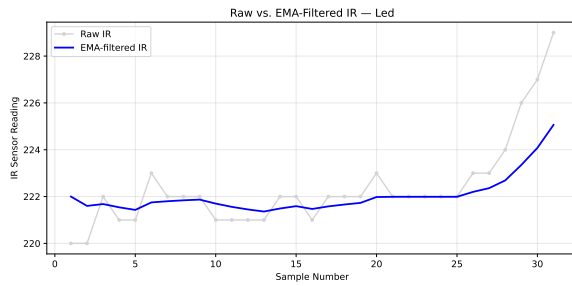


Figure 4: Raw and EMA-filtered IR readings under LED lighting.

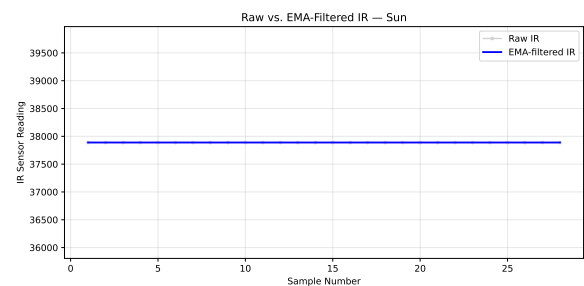


Figure 6: Raw and EMA-filtered IR readings under sunlight.

The raw readings moved up and down slightly.
The filtered readings were smoother and changed more gradually.
This shows that EMA reduced small fluctuations in the sensor measurements.

5.2 LED Lighting

Figure 4 shows the readings under LED lighting.
The raw readings showed small changes between samples.

The filtered readings were generally smoother.

The filter reduced small changes while still following the overall direction of the signal.

5.3 Fluorescent Lighting

Figure 5 shows the readings under fluorescent lighting.

This graph showed a larger change than the shade and LED graphs.

The raw IR reading dropped quickly near the end of the recording.

The EMA-filtered reading also decreased, but more slowly.

This happened because EMA uses the previous filtered value when calculating the new reading.

The result shows both an advantage and disadvantage of EMA.

It reduces sudden fluctuations, but it also introduces a delay when the actual sensor reading changes quickly.

5.4 Sunlight

Figure 6 shows the readings under strong sunlight.

Both the raw and filtered IR readings stayed at approximately 37889 in the recorded samples.

Because the readings were constant, the two graph lines overlap.

This graph does not show a reduction in noise because the recorded values did not change.

During the sunlight test, the UV sensor also reached a value of 4095.

This is the maximum value of the ESP32's 12-bit ADC.

Once the ADC reaches this value, it cannot show any further increase in the UV measurement.

The system still predicted sunlight, but the sensor had reached its measurement limit under these conditions.

6 Discussion

The main goal of this project was to build a classification model that could run directly on an ESP32.

The results showed that I could train the model on my computer and store it on the ESP32 using small lookup tables.

One of the main difficulties was getting DBSCAN to separate the four lighting conditions.

At first, the different sensor ranges affected how DBSCAN measured the distance between points.

The IR readings covered a much larger range, so using the original values did not give the clustering results I wanted.

After applying the logarithmic transformation and scaling both features, DBSCAN found four separate clusters with no noise points.

This showed how important the data preparation was. I did not have to change the main clustering method. I only had to improve how the measurements were prepared.

After clustering, I used KDE to model the four groups.

I then converted the KDE surfaces into small 8-bit tables that could be stored on the ESP32.

The four tables required only 1024 bytes, with another 16 bytes used for the scaling bounds.

This was much smaller than storing the same tables using floating-point values.

The stored model also gave the same predictions as the continuous KDE model on all 480 test samples.

The memory measurements showed that the complete program used 23% of the available flash and 7% of the available SRAM.

The classification time was approximately 3–6 microseconds.

These results show that the lookup table was a good choice for running the model on the ESP32.

The EMA graphs also showed how filtering affected the sensor readings.

For shade and LED lighting, the filtered values had fewer small fluctuations.

For fluorescent lighting, the raw readings dropped much faster than the filtered readings.

This showed that EMA can make a signal smoother, but it also introduces a delay when the signal changes suddenly.

The sunlight results showed another limitation.

The UV sensor reached the maximum ADC value of 4095 during strong sunlight.

Although the system predicted sunlight, the UV sensor could not measure any further increase beyond this value.

Finally, I compared ten ESP32 predictions with the Python model.

All ten results matched, which showed that the embedded version was producing the same predictions for those measurements.

Overall, the results showed that the system could classify lighting conditions while using a small amount of memory and processing time.

7 Limitations and Remaining Uncertainty

There are still some limitations with this project.

First, I collected data from only four lighting conditions.

Different bulbs, different brightness levels, and different sensor positions could produce readings that the model has not seen before.

Second, the training and test samples came from the same collection sessions.

Although the model achieved 100% accuracy on the held-out test set, this does not mean it will always be correct.

Testing on different days and in different locations would give a better idea of how well the model works outside the original collection conditions.

Another limitation is the EMA filter.

EMA reduced noise, but it also made the filtered readings respond more slowly to sudden changes.

The fluorescent graph showed this clearly.

Using a larger alpha value would make the filter react faster, but it would also allow more noise through.

The sunlight test showed a hardware limitation.

The UV sensor reached 4095, which is the maximum ADC value.

After reaching this limit, the system cannot measure a further increase in the signal using that ADC reading.

Another limitation comes from the feature scaling.

If a new measurement falls outside the original training range, the ESP32 clips it to the nearest edge of the grid.

This means that an unusual measurement could still receive a known class prediction.

The reject threshold can reject some weak matches, but it does not solve every case.

The 16×16 grid also has limited detail.

Using a larger grid could preserve more information from the KDE surfaces, but it would require more memory.

Finally, I only compared ten selected ESP32 measurements with the Python implementation.

Testing more readings would provide stronger evidence that the two implementations remain consistent.

8 Project Files and Demonstration

The source code, sensor data, Python scripts, trained model, and testing results are available in my GitHub repository:

```
https://github.com/Pinacam/  
classification-pipeline
```

The repository also contains a demonstration video showing the ESP32 making live predictions under different lighting conditions.

The video is saved in the `demo` folder. The repository also includes the CSV files used for training and testing, along with the graphs and generated model files.

9 Conclusion

In this project, I built a classification system that identified four lighting conditions using an ESP32, a UV sensor, and a TSL2591 light sensor.

I collected 2400 measurements and divided them into training, validation, and test sets.

I used DBSCAN to find groups in the sensor measurements.

After improving the feature scaling, DBSCAN found four clusters with no noise points.

I then used KDE to model each cluster and converted the results into small 8-bit lookup tables.

The final model classified all 480 held-out test samples correctly.

The stored lookup model also agreed with the continuous KDE model on all 480 samples.

The ESP32 and Python versions matched on ten selected sensor readings.

The complete Arduino program used 308800 bytes of flash and 23740 bytes of SRAM.

Classification took approximately 3–6 microseconds.

The EMA graphs showed that filtering reduced small fluctuations in the sensor readings, but also introduced a delay during larger changes.

The sunlight test showed that the UV sensor could reach its maximum reading under strong sunlight.

Overall, this project showed that I could train a model on my computer and run it directly on an ESP32 using small lookup tables.

For future improvements, I would collect more data under different lighting conditions, test the model on different days, and improve how the system handles out-of-range sensor readings.